UNIT 12 IMPLEMENTATION STRATEGIES-1

Structure **Page no.**

12.0 INTRODUCTION

You are aware of the object design models. Object design comes between the system design and implementation. After the designing stage, the implementation process gets start. Classes and their relationships which are developed during the object design; such designs are transformed into a particular programming language at the implementation stage. Generally, the task of converting an object design into source code is a straightforward process. But the feature such as the association of class diagrams in the design model is not possible to directly convert into programming language structures. Actually, most programming languages do not offer any paradigms to implement associations directly. This unit, explores some of the significant features and talks about the several strategies that should be adopted for their implementation.

This unit will explain concepts related to the implementation strategy, such as mapping design to code, and unidirectional and bi-directional associations. Association is a relationship between two separate classes through their objects. The implementation of association is unidirectional or bi-directional. Each association may be either optional, one–to–one, one–to–many associations. This Unit will also illustrate how to transform class definition from Class Diagram. This unit will also cover the important features and strategies for implementing unidirectional or bi-directional associations.

12.1 OBJECTIVES

After going through this unit, you should be able to explain:

- transforming design to code,
- explain different types of associations,
- how to create class definitions from class diagram,
- how the unidirectional associations are implemented,

- how the bidirectional associations are implemented, and
- difference between unidirectional and bidirectional associations.

12.2 MAPPING DESIGN TO CODE

In the software development cycle, when you reach the design part of the software, you create a design document, which includes different UML diagrams such as Design Class Diagram and Interaction Diagram, during the design model. This document is used as input to the code generation in object-oriented programming language.

Now, it is time to use the design document to start writing code. The mapping design to code means that design classes are converted into a program code by writing program statements in a programming language. So, classes in the design model can be implemented by writing program code in an object-oriented programming language such as C++ and Java etc.

A design class represents an abstraction of one or more classes in the software implementation. In fact, class and its objects depend only on the implementation language. For example, in an object-oriented language such as C++ and Java, a class can relate to a plain class. The design model can be very near to the implementation model, depending on how you draw its classes to classes or similar structures in the implementation language. The characteristics of the implementation language impact the design model, you must maintain the class structure as easy to understand and modifiable. The aim of implementation strategy is to use as a general framework for all object-oriented languages. Each programming language has its own number of features and syntax that are difficult to explain in a general process framework. So, you need to focus on the implementation approach so that you may use all potential features and handle the limitations of each programming language.

Mapping Designs to Code is a transformation process from UML class diagrams to class and interface definitions as well as method definitions (you will know more about this in Unit-2 of Block-4). Many UML tools have the facilities to generate code for the programming language. They can generate basic codes but cannot generate your ideas and interpretation of the diagram. Moreover, during object-oriented design modelling, you as a designer try to provide a great base for the code. In an object-oriented language, the implementation stage needs writing source code for class and interface definitions and method definitions.

The next section is related to the creating class definition from the designed class diagram.

12.3 CREATING CLASS DEFINITION FROM CLASS DIAGRAM

In the previous section, you learned to the need to transform a design model into code. This section defines how to transform class definition from a design class diagram.

A design class diagram helps you to write program code for software application development. Class diagram gives you an overview of your application software by depicting classes and their attribute, operations (methods) and relationships (dependencies, Generalization and Associations) among objects. You are aware of the essential elements of the class. This information is enough to create a basic class

definition in an object-oriented language. If you draw your class diagram with the help of UML software tool, it generates the basic class definition from the diagrams. There are many UML-oriented tools for transforming class diagrams into object-oriented code. For example if you are using 'NetBeans' IDE then you may add 'easyUML' plugin into 'NetBeans' to solve your purpose of basic implementation.

The following example illustrates how to create class definitions from the class diagram. Assume that the teacher wants to view the details of the students. Here is a partially created class diagram for Teacher and Student relationship as shown in figure-12.1. The diagram has two classes: Teacher and Student. Class Student defines only two attributes as rollno and name with getDetail() method. On the other side, class Teacher has only one method viewStudent().

Figure 12.1: Class diagram for Teacher and Student relationship

Following are the basic class definitions with attributes and method signatures of the above class diagram:

```
class Student
{
        String rollno;
        String name;
        public student(String rollno, String name)
        {
                this.rollno = rollno;
                this.name = name;
        }
        public String getDetail()
        {
        return(rollno+name);
        }
}
class Teacher
{
        public void viewStudent()      { ……          }
}
```

Now, you can add some more java code to the above example to complete the program. You can compile and run this program to check how this program works.

```
import java.util.Scanner;
public class Example
{
        public static void main(String[] args)
        {
        Student s1 = new Student("S123","xyz");
        Teacher t1 = new Teacher();
        t1.viewStudent(s1);
        }
}
class Student
{
        String rollno;
        String name;
        public Student(String rollno, String name)
        {
                this.rollno = rollno;
                this.name = name;
        }
        public String getDetail()
        {
         return(rollno+name);
        }
}
class Teacher
{
        public void viewStudent(student s)
        {
                Scanner scan = new Scanner(System.in);
                System.out.println("Enter Student Roll No");
                String getrollno = scan.nextLine();

                if(getrollno.equals(s.rollno))
                {
                System.out.println(s.getDetail());
                }
        }
}
```

12.4 IMPLEMENTING ASSOCIATIONS

Link and Association in UML representation are used for establishing relationships between objects and classes. A link is a logical or physical connection between two or more objects. For example, Ram *enrol-for* IGNOU University. A link is considered as an instance of an association. Association is a group of links that connect objects from the same classes. For example, students enrol-*for* a University. In programming languages, Association is implemented as a reference model in which one object is referenced from other objects, whereas links are not objects as they cannot be referenced but rely on the objects.

You can say that an association relates one or more object to one or more object of the other side. Associations are generally denoted as continuous lines between the participants' classes in the relationship. As shown in Figure 12.2, association is shown between Bank and Manager classes. An association is described by a name and the role played by objects of a class with respect to the other class and the multiplicity of each role. The multiplicity is specified by looking into how many objects of a class can be related to an object of the other class of the association.

The implementation of association is applied either in *unidirectional* or *bi-directional*. As the name implies that unidirectional is a one-way association, and bi-directional is a two-way association. Each association may be either optional, one–to–one, one–to–many association. For example, a teacher might be associated with a subject course such as one-to-one relationship, but a teacher may be associated with many students in the same course class indicates towards one-to-many relationship. If students in one section might be associated with students in another section of the same course, then it is a many-to-many relationship. In a similar manner, if all the sections of the course are related to a single course, then it is a many-to-one relationship.

The direction of traversal is important in association. Traversal means moving to other objects by following its reference. The direction of traversal is one-way from A to B if A only holds the reference to B. There can also be two-way direction where A and B both hold references to each other. Based on traversal, the association is a one-way or two-way association. The one-way or unidirectional association is traversed in one direction, and two-way or bi-directional association are traversed in both directions. If you wish to implement a unidirectional association, you may consider one point; then you shall maintain the association in only one direction. In the case of the implementation of bi-directional association, the association must be maintained in both directions. You can use pointers or association objects for implementing association. A pointer is an attribute that contains object reference, and association object is a pair of dictionary objects; one object is used for the forward direction and another for the backward direction.

The following example illustrates the concept of implementing association.

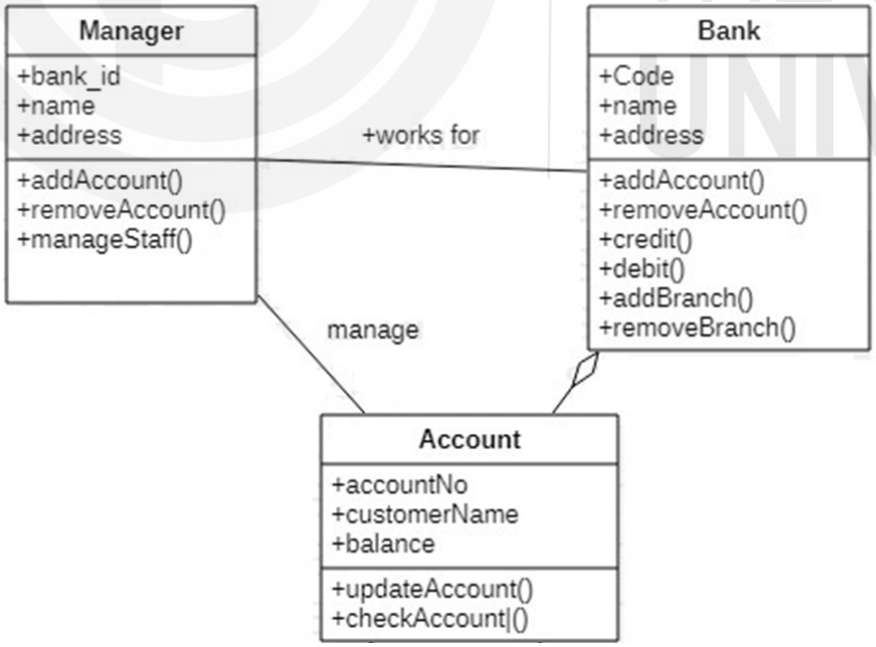

Figure 12.2: Relationship between Manager and Bank

The figure 12.2 defines classes such as Manager, Account, Bank and an association between Manager and Bank classes. Here, the Manager works for Bank and might manage many accounts of people in the Bank.

The implementation of one-way associations must be applied in one direction for the reason that a specific link wishes to be traversed in one direction only. This unidirectional association may be implemented through a single reference, pointing in the direction of traversal.

If one-to-one relationship exists between Bank and Manager, then the simple pointer is used, as shown in Figure-12.3; if there is one-to-many relationship then multiple pointers are used for implementing unidirectional or one-way association.

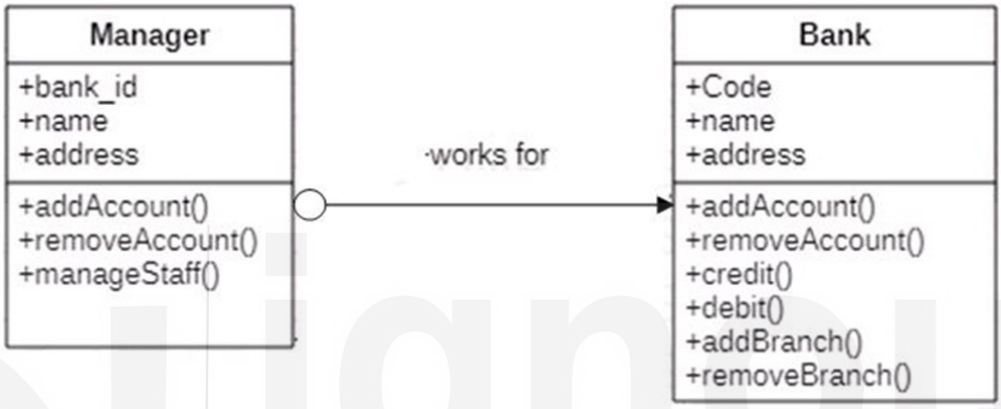

Figure 12.3: Implementation of unidirectional association using pointers

As far as bi-directional associations, they traverse in both directions. Generally, such associations do not traverse with equal frequency in both directions. If bi-directional association traverse with more frequencies in both directions, attributes should be implemented using a set of pointers, as shown in Figure-12.4. This approach provides fast access, but you may have to update attributes in both directions to maintain link consistency. It means that if one attribute is updated on one side, then the attribute on the other side must be updated. If traversal is in less frequency, then implement as an association in one direction only.

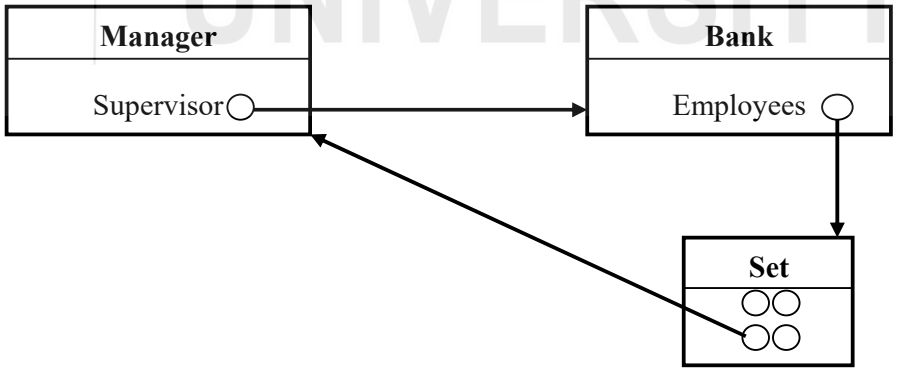

Figure 12.4: Implementation of bi-directional association using pointers

Usually, the two distinct aspects of the implementation of associations are as under:

- The data declaration is essential to define so that it permits the details of actual links to be stored. It consists of defining data members in one class which can store references to objects of the associated class.

- It is required to consider that pointers can be manipulated by the rest of the application. The association's underlying implementation details should be unseen from the client code.

You will study more about the unidirectional implementation in section 12.5 of this unit ,and bi-directional implementation is in section 12.6 of this unit.

☞ Check Your Progress - 1

1. Explain the Mapping of Designs to Code in detail.

   ---

2. What is the benefit of designing class diagram?

   ---

3. Explain the two distinct aspects of the implementation of associations?

   ---

4. What do you understand by the term 'Association' in UML? Discuss the types of associations. How does Association differ from Link?

12.5 UNIDIRECTIONAL IMPLEMENTATION

This section will describe cases that help you understand unidirectional implementation of association. This type of association will be supported in one direction only. It means that one class object is linked to another object of associated class in one direction. The direction of traversal of an object can be represented in a class diagram by putting an arrow on the association.

The following subsections define the unidirectional implementation of association for different multiplicity is as follows:

- Optional,

- One-to-one , and
- Association with Multiplicity

12.5.1 Optional Association

In optional association, a link between the participating objects may or may not exist. This type of association is implemented in one direction only. The figure 12.5 shows an optional association between two classes as Person and Credit Card. Every person may or may not have a credit card. It only depends upon the person's need, who desires to take a credit card or not.

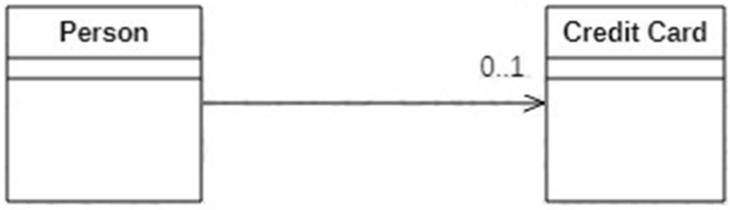

Figure 12.5: Example of Optional Association

The implementation of optional association is easy. You can implement optional association through a simple reference variable, as revealed in the following code. This allows 'Person' object to encompass a reference to one credit card object. If a person is not associated with a card, then the reference variable will be null. The implementation code is listed below:

```
public class Person
{
        private CreditCard cCard

        public CreditCard getCard()
        {
        return cCard;
        }

        public void setCard(CreditCard card)
        {
        cCard = card;
        }

        public void removeCard()
        {
         cCard = null;
        }
}
```

12.5.2 One-to-One Association

The one-to-one association is a simplest form of association to implement between two classes. In one-to-one association, one occurrence of a class is related to exactly one occurrence of the associated class. In unidirectional association, you can traverse from an occurrence of one class to an occurrence of the associated class (as indicated by the direction of the arrow) but not in a reverse direction. This type of association is easily implemented by using an attribute that contains a reference to an occurrence of

the associated class. For example, one-to-one association exists in two classes: Clock and Display, as shown in figure 12.6.

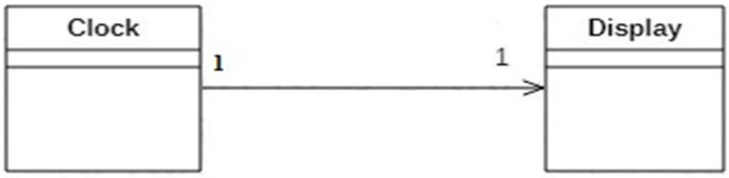

Figure 12.6: Example of One-to-one Unidirectional Association

The implementation of the above figure is as under:

```
public class Clock
{
        private Display theDisplay;
        ……..
        ……
}
```

Another example illustrates one-to-one Unidirectional Association. You must know that each person has only one saving account in a bank. So, this is a One-to-one Association. Following figure-12.7 displays one–to–one relationship between Person and Bank. This figure demonstrates two classes: Person and Bank.

Figure 12.7: A One-to-one Unidirectional Association

Each account is associated with exactly one person, and link is traversed from Person class to associated Account class. In this situation, Account is created by the Person constructor. Person class holds a link to the Account Class but not in a reverse direction. Now, you can subsequently write code if the above assumptions are taken into consideration. The implementation of the above figure is as under:

```
public class Person
{
  private Account account;

   public Person()
        {
                account = new Account();
        }
 public Account  getAccount()
        {
                return account;
        }
}
```

12.5.3 Association with Multiplicity 'Many'

This type of association exists in those cases where one instance of a class is related to many instances of associated class. You might be seen that every department has many employees. This is an example of one-to-many association between department and employees. In essence, it is a one-to-many association in which data flows from one to many directions.

You may take another example on teacher and student. In a school, each teacher is responsible for teaching a number of students. A teacher object could be associated with many student objects in this association. Here, you can use multiple pointers to implement this association. Each pointer is linked to each student. The teacher class provides operations to maintain the collection of these pointers. Existence of one link must be required between a teacher and student class for implementing this association as shown in figure-12.8. In addition, a data structure is also required for storing all pointers.

Figure 12.8: An association with multiplicity 'many'

This type of association can be implemented using a suitable container class. The class affirms a vector of students as a private data member and the methods to add/remove students from this collection by simply calling the corresponding methods defined in the interface of the vector class.

```
public class Teacher
{
   private Vector stuobj
         public void addStudent(Student stu)
         {
              stuobj.addElement(stu) ;
         }
         public void removeStudent(Student stu)
         {
         stuobj.removeElement(stu) ;
          }
}
```

☞ Check Your Progress - 2

1. Explain the unidirectional relationship? Describe how the one-to-one associations can be implemented.

---

2. What is an optional association? Explain the implementation of optional association with an example.

---

3. Explain one-to-one uidirectional association with an example in favour of implementation.

12.6 BI-DIRECTIONAL IMPLEMENTATIONS

In the previous section, you have studied the unidirectional implementation of associations where you can navigate in one direction but cannot navigate in the reverse direction. This section explaines bidirectional implementation of association. Bi-directional relationship provides navigational access in both directions, so that you can access the entity of the other side without explicit requests. Links in both directions are required to be maintained for implementing bi-directional associations.

In a bidirectional implementation, each entity has a relationship property or field that refers to the other entity. Through the relationship property or field, an entity of class can access the related object of associated class. If an entity has a related field, the entity is said to "know" about its related object.

A bidirectional association is usually more difficult to maintain than a unidirectional association. It requires additional program code for accurately creating and deleting the relevant objects. This creates a more complex program. Apart from this, an improperly implemented bidirectional association can cause problems for garbage collection of unused objects.

A pair of references are needed for implementing each link in bi-directional implementation of an association. The essential program codes needed to support such implementations are the same as required in the unidirectional implementation. The only variance is that appropriate fields have to be declared in both classes which are taking part in the bi-directional association.

The following subsections define bi-directional implementation of association for different multiplicity is as follows:

- One-to-one and Optional Associations ,
- One-to-many , and
- Immutable Association

12.6.1 One-to-One and Optional Associations

In One-to-one bi-directional association, you can traverse from an occurrence of one class to an occurrence of the associated class and vice versa. The figure-12.9 demonstrates a bidirectional association in which not any arrow is used on any end of the association. You must be able to traverse from an Account class to the corresponding Person class and vice-versa.

Figure 12.9: Example of one-to-one bi-directional association

For implementation, you have to create two classes: Person and Account. Both the classes contain a link through which it is associated. Each class needs an attribute that contains a reference to an object of the associated class. Each class must have a constructor. The account attribute is initialized in the constructor of the Person class, and it is never modified. In a similar manner, person attribute is initialized in the constructor of the Account class, and it is also never modified. Consider the following implementation code in java for One-to-one bi-directional association.

```
public class Person
 {
    private Account account;
          public Person()
          {
          account = new Account(this);
          }
          public Account getAccount()
          {
          return account;
          }
 }

 public class Account
{
   private Person person;
   public Account(Person person)
   {
          this.person = person;
   }
public Person getPerson()
         {
          return person;
          }
}
```

12.6.2 One-to-Many Association

In One-To-Many associations, one source is connected with multiple targets. For example, many students can register for one programme. In One-to-many bi-

directional association, you can traverse from an occurrence of one class to many occurrences of the associated class and vice versa.

Consider the following example; here we have models two classes: Person and Account. There is a One-to-Many relationship between them, which means that one Person has many Accounts as shown in figure 12.10.

Figure 12.10: A one-to-many bi-directional association

For implementing this association, the Person class can hold a data member to store a set of pointers to Account class. In addition, each account should store a single pointer to a 'Person' class. The responsibility of maintaining the links shall be given to the Person class.

In Object Orientated programming languages, One-to-one relationships are generally implemented with collection objects such as a Set, List or Map, or even a simple array. Java libraries provide collection classes such as HashSet, ArrayList and HashMap, which implement the Set, List and Map interfaces. Following implementation code in java for One-to-many bi-directional association are given:

```
public class Account
{
        private Person person;
        public void setPerson( Person newperson)
         {
                if (person != newperson)
                {
                 Person oldperson = newperson;
                 person = newperson;
                }
                if (newperson != null)
                {
                newperson.addAccount(this);
                }
                if (oldperson != null)
                {
                 oldperson.removeAccount(this);
                }
        }
}
public class Person
{
        private Set accounts;
        public Person()
        {
        accounts = new HashSet();
        }
        public void addAccount(Account ac)
        {
                accounts.add(ac);
                ac.setPerson(this);
        }
        public void removeAccount(Account ac)
        {
                accounts.remove(ac);
                ac.setPerson(null);
         }
}
```

12.6.3 Immutable Association

In object-oriented programming, an immutable object is unchangeable object, and its state cannot be modified after it is created, whereas mutable objects are changeable objects which can be modified after their creation. Immutable objects are inherently thread-safe and offer higher security than mutable objects. Associations are immutable data structures. This means that they carry no state and a copy of an Association is another completely independent Association.

Consider the following example for Immutable Association. An association between account and debit card are traversed in both directions. It is assumed that there is a restriction for the debit card; it can only be issued to person per account. The relevant class diagram which preserves this restriction is shown in figure 12.11.

Figure 12.11: An immutable one-to-one association

Each class defines a data member with reference to an associated class object. The actual code of association is concluded at the time of defining the requirements of the system. The implementation of this association is described as follows:

```
public class Account
{
     private DebitCard DbCard;

          public Account(DebitCard  dcard)
          {
          DbCard = dcard;
           }

          public DebitCard  getDebitCard ()
          {
           return DbCard;
          }
 }

public class DebitCard
{
          private Account theAcc ;

          public DebitCard (Account ac)
```

☞ Check Your Progress - 3

1. What is bi-directional relationship? How the bi-directional Implementations are made.

---

2. What is the difference between unidirectional and bidirectional association? Explain an approach to implementing bidirectional association.

---

3. How is a one-to-one bi-directional association different from one-to-many bi-directional association? Explain how one-to-one bi-directional association is implemented with the help of an example.

12.7 SUMMARY

In this unit, you have learned mainly about implementation strategy, which transforms design into code in a programming language. You have also learned how to create a basic class definition from the design class diagram created during the software development phase. A design class diagram helps you write program code for software application development and gives you an overview of your application software by depicting classes and their attributes, operations and relationships among objects. Each designed class is transformed into one or more classes; it only depends on the programming language.

Association creates a relationship between two separate classes through their objects. This relationship can be unidirectional or bi-directional. This unit also explained the implementing association. There are some decision rules for implementing association, such as in unidirectional association; the decision has been taken into account for maintaining the association in only one direction, while in bi-directional implementations, association must be maintained in both directions. Each association may be either one-to-one, one-to-many or many-to-many relationships. Association represents static relationships between classes.

12.8 SOLUTIONS/ ANSWER TO CHECK YOUR PROGRESS

☞ Check your Progress - 1

1. The mapping design to code means that design classes are converted into a program code by writing program statements in a programming language. Classes in the design model are implemented by writing program code in an object-oriented programming language such as C++ and Java. Mapping Designs to Code is a transformation process from UML class diagrams to class definitions and method definitions. Many UML tools have the facilities to generate code for the programming language. If you draw your class diagram with the help of UML tool, it can generate the basic class definition from the diagrams. There are many UML-oriented tools for transforming class diagrams into object-oriented code.

2. A designed class diagram helps you to write program code for software application development. Class diagrams give you an overview of your application software by depicting classes and their attribute, operations (methods) and relationships (dependencies, Generalization and Associations) among objects. This information is enough to create a basic class definition in an object-oriented language. If you draw your class diagram in a UML tool, it can generate the basic class definition from the diagrams.

3. The two distinct aspects of the implementation of associations are as under:

   - The data declaration is essential to define that it permits the details of actual links to be stored. It consists of defining data members in one class which can store references to objects of the associated class.
   - It is required to consider that the rest of the application can manipulate pointers. The association's underlying implementation details should be unseen from the client code.

4. Association creates a relationship between two separate classes through their Objects. This relationship can be unidirectional or bi-directional. Each association may be either one-to-one, one-to-many and many-to-many relationships.

   - One-to-One: In “one-to-one” relationship, one object is related to exactly one object of associated class. For example, a person can have only one VoterID and this ID can be linked to only one person.
   - One-to-many: A University can have many Students. So it is a “one-to-many” relationship.
   - Many to one: Many students may enrol in a single course which is a “many-to-one” relationship.

   Link and Association in UML illustration are used for establishing relationships between objects and classes. A link is a logical or physical connection between two or more objects. For example, Ram enrols at IGNOU University. A link is considered as an instance of an association. Association is a group of links that connect objects from the same classes. For example, students enrol-for a University. In programming languages, association is implemented as a reference model in which one object is referenced from other object, whereas links are not objects as they cannot be referenced but rely on the objects.

☞ Check your Progress - 2

1. Unidirectional relationship defines only one entity with a relationship property that references the other object. For example, People own book(s). It navigates only in one direction but cannot navigate in the reverse direction.

   The association properties can be implemented directly by providing appropriate data declaration in the associated classes. Other semantic features of an association can be set by providing only a limited range of operations in the class's interface or by involving code in the implementation of member functions which confirms that the necessary constraints are maintained.

2. In optional association, a link between the participating objects may or may not exist. This type of association is implemented in one direction only. For example, optional association exist between the person and credit card (see figure-12.5). Every person may or may not have a credit card. It only depends upon the person's need, who desires to take credit card or not. The implementation of optional association is easy. Optional association can be implemented by using a simple reference variable. This permits to person object to contain a reference to one credit card object. If a person is not associated with a card, then the reference variable will be null.

3. The one-to-one association is a simplest form of association to implement between two classes. In One-to-one Association, one occurrence of a class is related to exactly one occurrence of the associated class. In unidirectional association, you can traverse from an occurrence of one class to an occurrence of the associated class but not vice-versa. This type of association is easily implemented by using an attribute that contains a reference to an occurrence of the associated class. For example, one-to-one association is exist in two classes as Clock and Display as shown in following figure.

Figure 12.12 : One to One Association

The implementation of the above figure 12.12 is as under:

```
public class Clock
{
        private Display theDisplay;
        ……..
        ……
}
```

☞ Check your Progress - 3

1. Each entity in a bidirectional relationship has a relationship property that refers to the associated entity. Through the relationship property, an entity of class can access related objects of associated class. If an entity has a related field, the entity is said to "know" about its related object. A bidirectional

association is usually more difficult to maintain than a unidirectional association. It requires additional program code for accurately creating and deleting the relevant objects. This creates a more complex program. Apart from this, an improperly implemented bidirectional association can cause problems for garbage collection of unused objects. A pair of references are needed for implementing each link in bi-directional implementation of an association. The essential program codes which are needed to support such implementations are the same as required in the unidirectional implementation. The only variance is that appropriate fields have to be declared in both classes which are taking part in the bi-directional association.

2. The bidirectional association provides navigational access in both directions, whereas unidirectional association only provides navigation in one direction. By using bidirectional association, you can access the other side without explicit request, and it also allows you to apply cascading options to both directions.

3. In one-to-one bi-directional association, one occurrence of a class is related to exactly one occurrence of associated class, while in one-to-many bi-directional association, one occurrence of a class is related to many occurrences of the associated class. For example, a teacher might be associated with a subject course (a one-to-one relationship) but also with each student in the same course class (a one-to-many relationship).

12.9 REFERENCES/FURTHER READING

- James Rambaugh, Michael Blaha, William Premerlam, Frederick Eddy, William Corensen, “Object Oriented Modelling and Design” PHI, New Delhi, 2004.
- https://www.uml-diagrams.org/index-examples.html
- https://www.uml-diagrams.org/class-diagrams-overview.html
- https://netbeans.org/features/uml/
- http://plugins.netbeans.org/plugin/55435/easyuml
- https://www.uml-lab.com/en/uml-lab/tutorials/modellierung-und-codegenerierung/